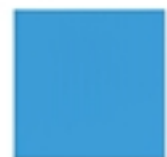
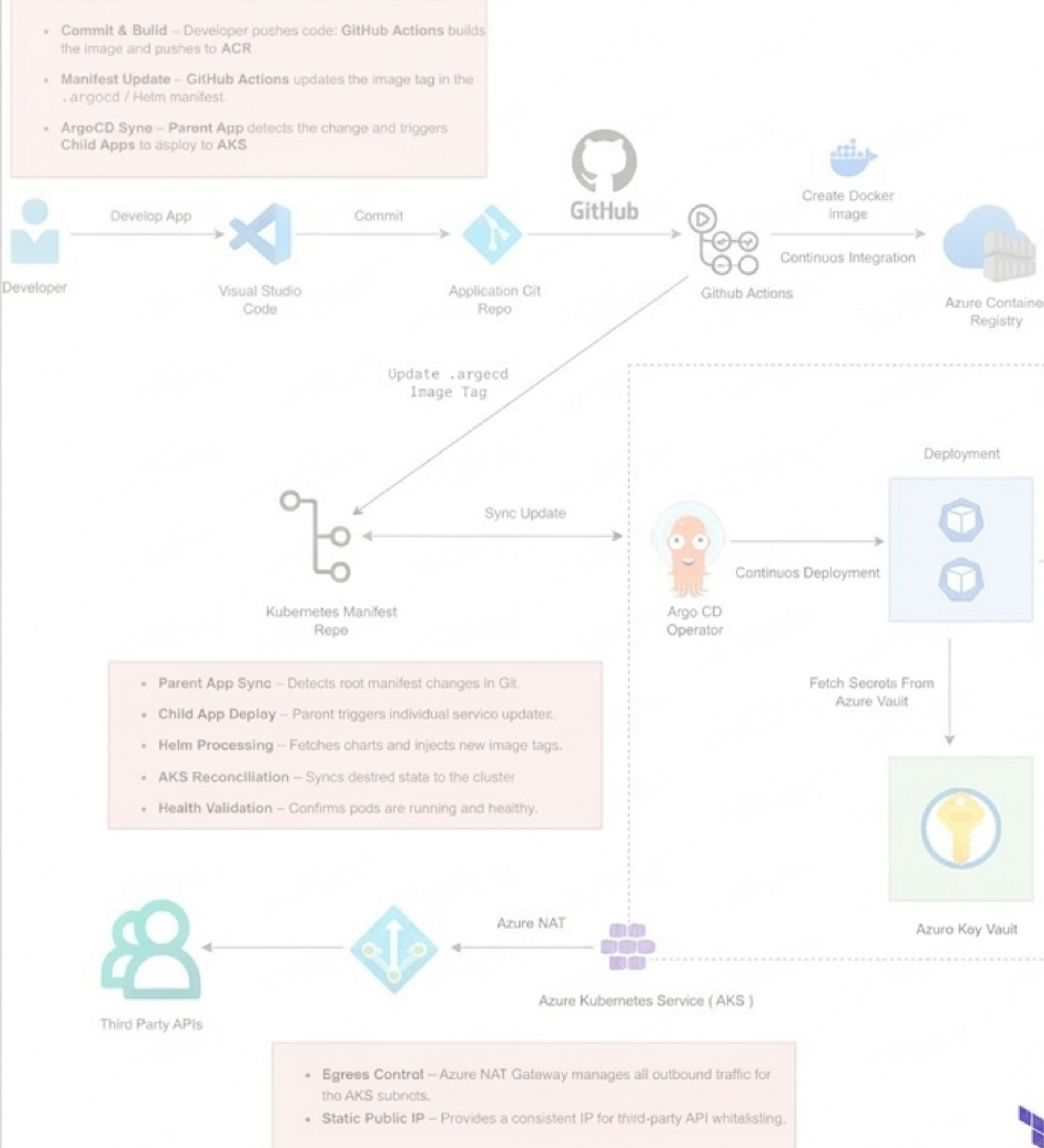


Day 1: Infrastructure Foundations

Architecture, Repositories, and Network Strategy

-  1. Master Architecture Breakdown
-  2. The 3-Repository Strategy
-  3. Terraform Folder Structure
-  4. Configuring Remote State
-  5. The 2-VNet Network Foundation



Flow 1: App Dev & CI
GitHub Actions -> Azure Container Registry

Flow 2: Continuous Deployment
ArgoCD -> AKS Cluster

Flow 3: Infrastructure Provisioning
Terraform -> Azure Resources

Day 1 Objective: We are ignoring the application and CI/CD layers for now to strictly build the foundation—Terraform provisioning the Azure base.

terraform-azure

Function:

Pure Azure resource provisioning via Infrastructure as Code.

The Why:

Isolates infrastructure state from application code. Prevents application updates from accidentally triggering infrastructure destruction.

kubernetes

Function:

ArgoCD configurations, Helm charts, and manifest files.

The Why:

Keeps application deployments decoupled from the underlying compute hardware, ensuring smooth GitOps workflows.

azure-vault

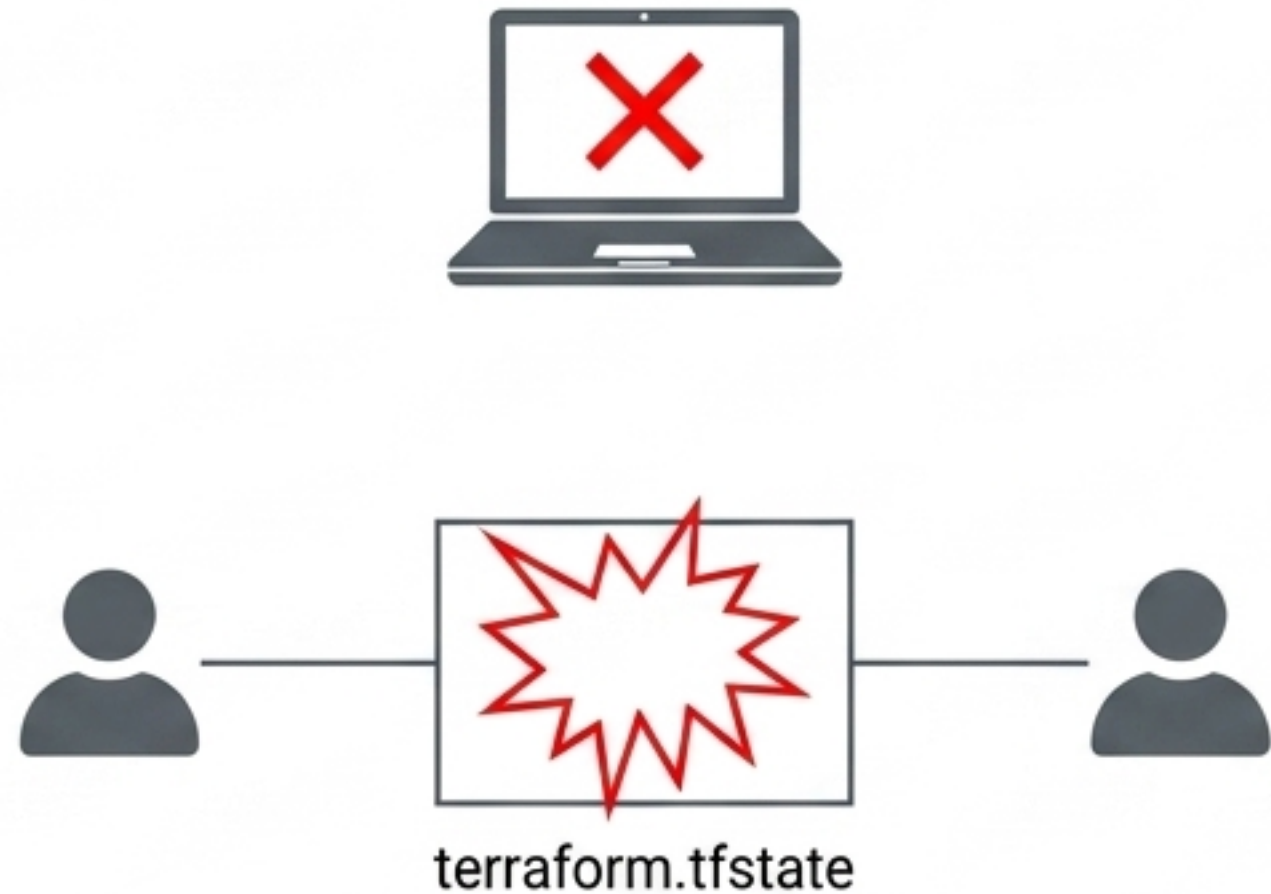
Function:

Dedicated repository for Azure Key Vault and secret management.

The Why:

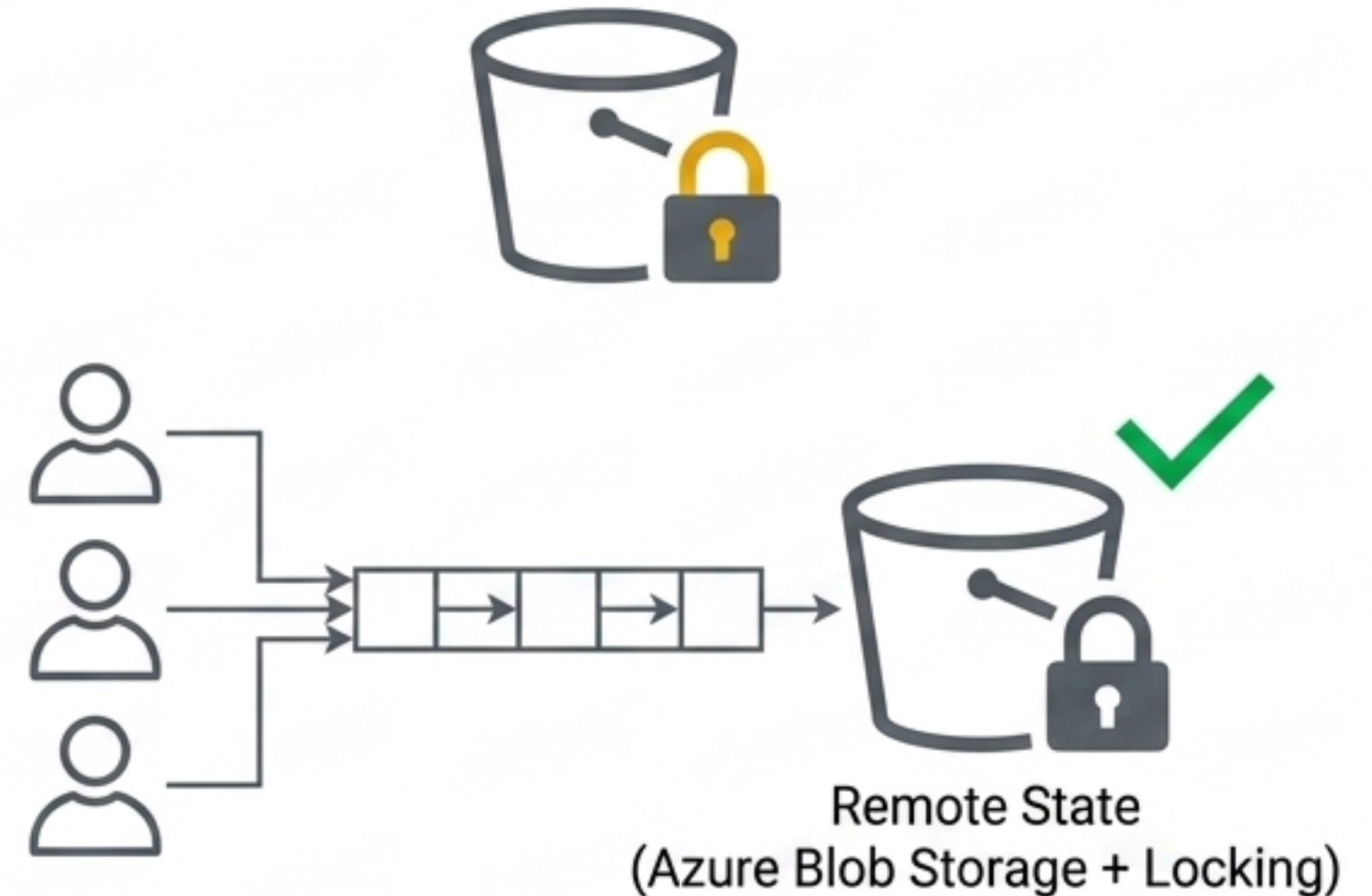
Establishes a strict security boundary. Secrets must never live alongside application code or standard infrastructure state.

THE RISK



Local state creates a single point of failure, prevents team collaboration, and risks catastrophic state file corruption.

THE STANDARD



Remote State (Azure Blob Storage + Locking).
Provides a synchronized Single Source of Truth, enables safe team collaboration, and ensures disaster recovery.

Remote state is not an optional feature—it is a mandatory prerequisite for production-grade DevOps.

00_resource_groups/

- rg.tf
- variables.tf
- providers.tf
- outputs.tf

01_networking/

- vnet.tf
- nat.tf
- peering.tf
- variables.tf
- providers.tf

Layer 1: 01_networking

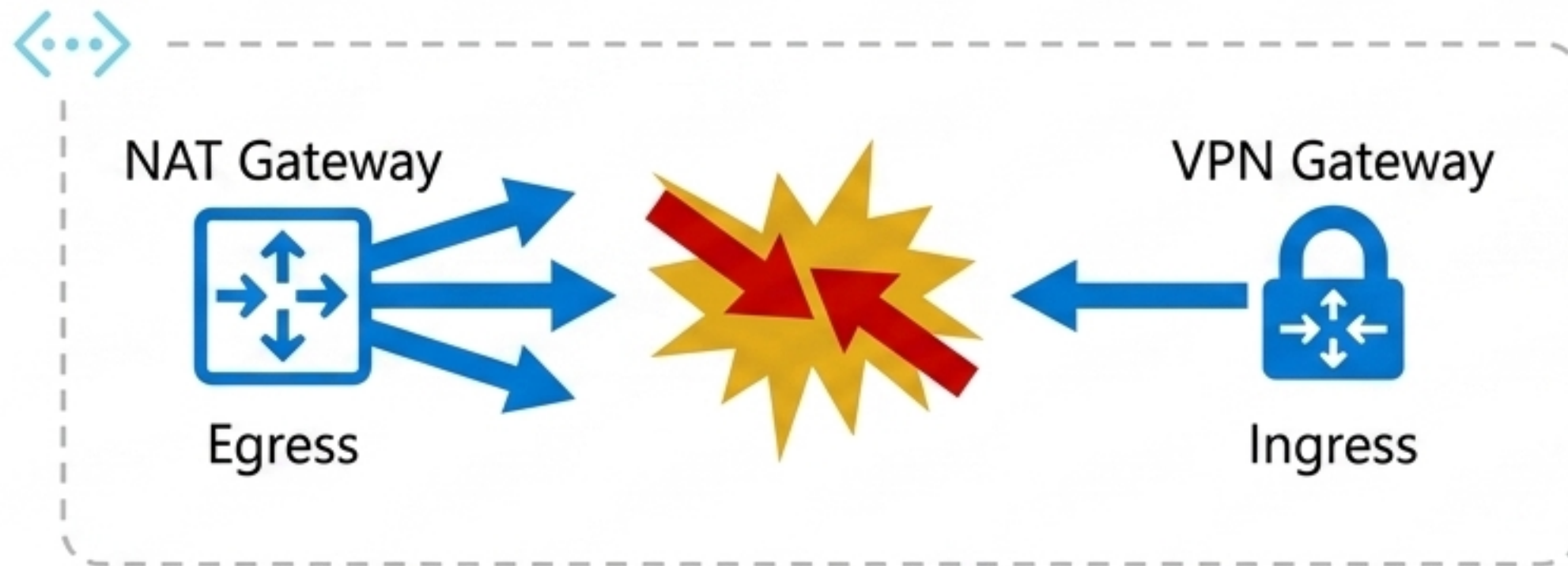
The plumbing. Deploys inside Layer 0 to establish communication boundaries.

Layer 0: 00_resource_groups

The container. Nothing can exist without a Resource Group to hold it.

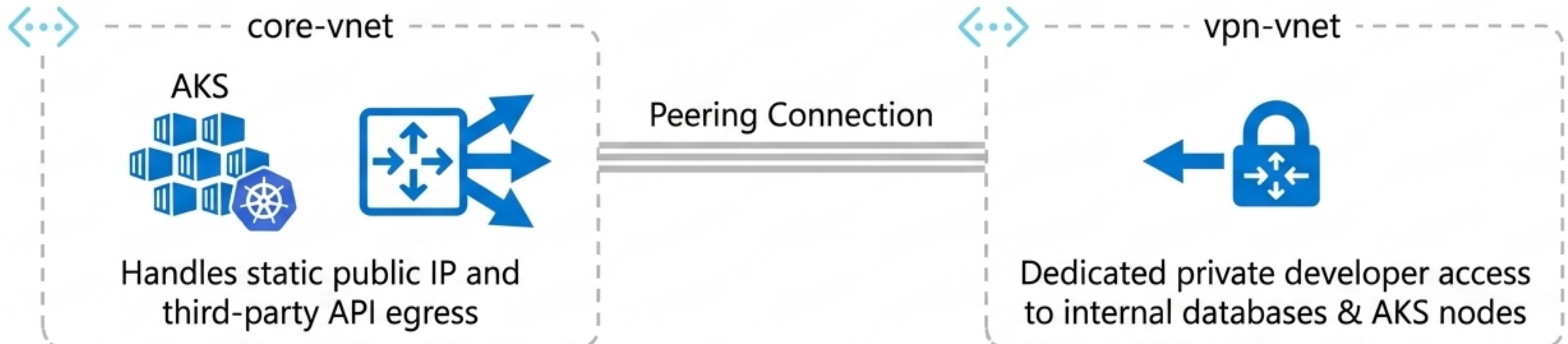
Splitting configurations into numbered execution folders prevents massive, unmanageable monolith state files and enforces a logical, dependency-driven deployment order.

THE PROBLEM

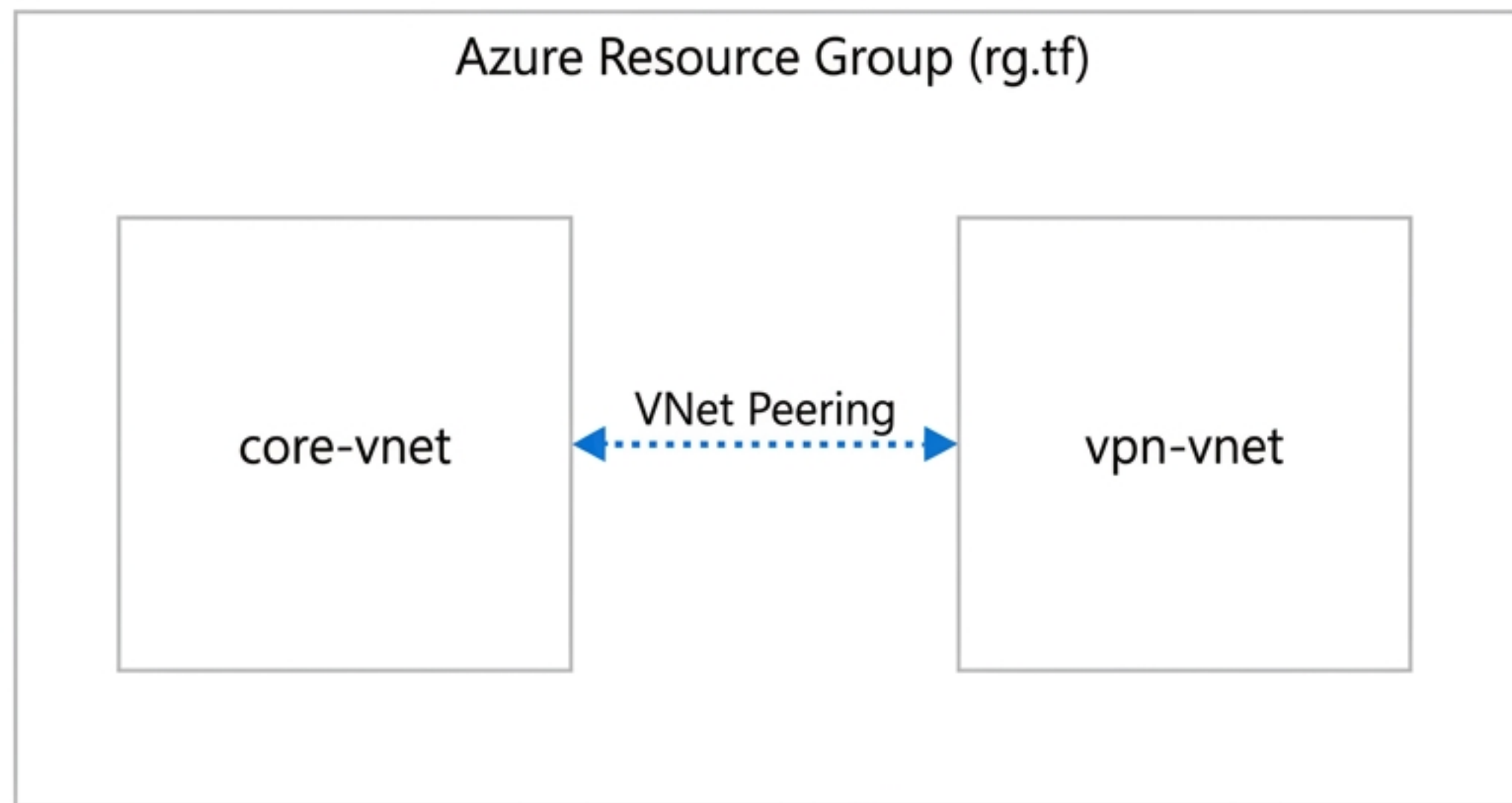


A NAT Gateway governs all outbound traffic for AKS subnets, making it difficult and error-prone to configure private VPN access within the same boundary.

THE SOLUTION



01_networking folder



Execution Note: The 01_networking folder must handle both the creation of the isolated networks and the explicit peering connection that allows them to communicate.

- ☒ **Analyzed the Master Blueprint:** Extracted the core infrastructure layer from the application layer.
- ☒ **Established Boundaries:** Created 3 strict repositories (terraform, kubernetes, vault).
- ☒ **Secured Terraform:** Configured remote state to eliminate local collision risks.
- ☒ **Structured for Scale:** Built layered 00_resource_groups and 01_networking folders.
- ☒ **Resolved Network Conflicts:** Designed and deployed the 2-VNet (Core + VPN) peering strategy.

Day 1 Complete. You are no longer just running tutorials—you are engineering solutions with intent.